WEEK – 9.1

CAPSTONE PROJECT: EARTHQUAKE ALERT PREDICTION

DATA COLLECTION AND DATA ANALYSIS:

The dataset is downloaded from Kaggle:
https://www.kaggle.com/datasets/warcoder/earthquake-dataset

The dataset contains earthquake recorded from the year 1-Jan-2001 to 1-Jan 2023, containing total of 782 rows and 19 columns. Each columns are the geological parameters, given below:

- title: Name given to the earthquake
- magnitude: The magnitude of the earthquake
- date_time: Date and time on which earthquake occurred
- cdi: (Community Decimal Intensity) The maximum reported intensity for the event range
- mmi: (Modified Mercalli Intensity) The maximum estimated instrumental intensity for the event - scaled and ranked from 1 to 12
- alert: The alert level- 'green', 'yellow', 'orange', 'red'.
- tsunami: '1' for events in oceanic regions and '0' otherwise
- sig: A number describing how significant the event is. Larger numbers indicate a more significant event. The value is determined on a number of factors, including: magnitude, maximum mmi, felt reports, and estimated impact
- net: The ID of the data contributor. Identifies the network considered to be the preferred source of information for the event
- nst: The total number of seismic stations used to determine earthquake location.
- dmin: Horizontal distance from the epicentre to the nearest station
- gap: The largest azimuthal gap between azimuthally adjacent stations (in degrees). In general, the smaller this number, the more reliable is the calculated horizontal position of the earthquake. Earthquake locations in which the azimuthal gap exceeds 180 degrees typically have large location and depth uncertainties
- magType: The method or algorithm used to calculate the preferred magnitude for the event
- depth: The depth where the earthquake begins to rupture
- latitude/ longitude: Coordinate system by means of which the position or location of any place on earth's surface can be determined and described
- location: Location within the country
- continent: Continent of the earthquake hit country
- country: Affected country.

From the above parameters, we have two dependent variables: 'alert' and 'tsunami'. Since it is about Earthquake alert prediction, 'tsunami' variable is omitted and retain 'alert' variable for this project.

The dataset is screened for pre-processing to eliminate null values and remove some independent variables that doesn't serve best for the earthquake alert prediction.

The 'alert' parameter has 367 null values, 'continent' with 576 null values and 'country' with 298 null values. Since the 'alert' parameter categorization is dependent on various parameters along with 'magnitude', it is not possible to predict null values for 'alert' parameter. Hence we drop all the null values from the 'alert' parameter. The 'continent' parameter along with 'title', 'date_time', 'tsunami', 'net', 'magType', 'latitude', 'longitude', 'location', 'nst' are removed from the dataset.

The qualitative and quantitative data from the dataset are separated and checked for the outliers. In quantitative data, 'magnitude', 'sig', 'dmin', 'gap', 'depth' contains Greater than Outliers. Since the raw data is recorded from the natural calamity, the data may naturally contain uncertain range of values, which cannot be categorized as Greater than Outlier, because every range of values are required for accurate prediction.

The further analysis, is done based on the following questions:

1. Which country has the highest rate of earthquake occurrence?
2. What is the minimum and maximum magnitude of the highest earthquake occurring area?
3. How many red alert did the highest earthquake occurring region get?
4. Which country has the lowest rate of earthquake occurrence?
5. What is the minimum and maximum magnitude of the lowest earthquake occurring area?
6. How many red alert did the lowest earthquake occurring region get?
7. Top 10 earthquake occurring places?
8. Which country has recorded the highest magnitude of earthquake?
9. Which countries receives more red alert?
10. Magnitudes of red alert and it's region
11. Which data field are highly correlated with 'magnitude'?
12. Which data field are least correlated with 'magnitude'?

The results and code for data analysis is in the following link:
https://github.com/AuxiliaV/Week-9.1_MachineLearningAndDataScienceCapstoneProject/blob/main/Earthquake%20Alert%20Prediction-Data%20Preprocessing%20and%20Analysis.ipynb

FEATURE SELECTION:

To identify the significant features, the dataset is subjected to 4 feature selection algorithms: Select K Best, Feature Importance, Forward Selection and Backward Elimination. The feature selection is done on 'magnitude', 'sig', 'dmin', 'gap', 'depth'. Two features: 'cdi' and 'mmi' are removed for feature selection, as it is collected after the occurrence of earthquake. The given below results displays the three significant selected features followed by its scores.

Github: https://github.com/AuxiliaV/Week-9.1_MachineLearningAndDataScienceCapstoneProject/blob/main/Earthquake%20Alert%20Prediction%20-%20Feature%20selection.ipynb

1. **Select K Best:**
   - For n = 3

```
['sig', 'gap', 'depth']
```

	Logistic	SVML	KNN	NaiveBayes	DecisionTree	RandomForest
ChiSquare	0.798077	0.798077	0.826923	0.778846	0.759615	0.807692

   - For n = 4

```
['sig', 'dmin', 'gap', 'depth']
```

	Logistic	SVML	KNN	NaiveBayes	DecisionTree	RandomForest
ChiSquare	0.798077	0.798077	0.817308	0.769231	0.788462	0.817308

   - For n = 5

```
['magnitude', 'sig', 'dmin', 'gap', 'depth']
```

	Logistic	SVML	KNN	NaiveBayes	DecisionTree	RandomForest
ChiSquare	0.807692	0.807692	0.778846	0.759615	0.769231	0.788462

2. **Feature Importance:**
   - For n = 3

```
['sig', 'dmin', 'depth']
['sig', 'depth', 'gap']
```

	Logistic	SVML	KNN	NaiveBayes	DecisionTree	RandomForest
RandomForest	0.788462	0.798077	0.798077	0.826923	0.769231	0.807692
DecisionTree	0.798077	0.798077	0.826923	0.778846	0.759615	0.798077

   - For n = 4

```
['sig', 'dmin', 'depth', 'magnitude']
['sig', 'depth', 'gap', 'dmin']
```

	Logistic	SVML	KNN	NaiveBayes	DecisionTree	RandomForest
RandomForest	0.807692	0.817308	0.817308	0.798077	0.769231	0.798077
DecisionTree	0.798077	0.798077	0.817308	0.769231	0.778846	0.817308

   - For n = 5

```
['sig', 'dmin', 'depth', 'magnitude', 'gap']
['sig', 'depth', 'gap', 'dmin', 'magnitude']
```

	Logistic	SVML	KNN	NaiveBayes	DecisionTree	RandomForest
RandomForest	0.807692	0.807692	0.778846	0.759615	0.798077	0.798077
DecisionTree	0.807692	0.807692	0.778846	0.759615	0.807692	0.807692

3. **Forward Selection:**
   - For n = 3

```
Selected Feature Names:
('magnitude', 'sig', 'gap')
DecisionTreeClassifier(max_features='sqrt', random_state=0)
Selected Feature Names:
('magnitude', 'sig', 'depth')
```

	Logistic	SVML	KNN	NaiveBayes	DecisionTree	RandomForest
RandomForest	0.817308	0.807692	0.788462	0.778846	0.807692	0.836538
DecisionTree	0.798077	0.817308	0.817308	0.788462	0.798077	0.826923

- For n = 4

```
Selected Feature Names:
('magnitude', 'sig', 'gap', 'depth')
DecisionTreeClassifier(max_features='sqrt', random_state=0)
Selected Feature Names:
('magnitude', 'sig', 'gap', 'depth')
```

	Logistic	SVML	KNN	NaiveBayes	DecisionTree	RandomForest
RandomForest	0.807692	0.817308	0.788462	0.759615	0.798077	0.807692
DecisionTree	0.807692	0.817308	0.788462	0.759615	0.798077	0.807692

- For n = 5

```
Selected Feature Names:
('magnitude', 'sig', 'dmin', 'gap', 'depth')
DecisionTreeClassifier(max_features='sqrt', random_state=0)
Selected Feature Names:
('magnitude', 'sig', 'dmin', 'gap', 'depth')
```

	Logistic	SVML	KNN	NaiveBayes	DecisionTree	RandomForest
RandomForest	0.807692	0.807692	0.778846	0.759615	0.769231	0.788462
DecisionTree	0.807692	0.807692	0.778846	0.759615	0.769231	0.788462

4. **Backward Elimination:**
   - For n = 3

```
Selected Feature Names:
('magnitude', 'sig', 'depth')
DecisionTreeClassifier(max_features='sqrt', random_state=0)
Selected Feature Names:
('magnitude', 'sig', 'depth')
```

	Logistic	SVML	KNN	NaiveBayes	DecisionTree	RandomForest
RandomForest	0.817308	0.807692	0.788462	0.778846	0.807692	0.836538
DecisionTree	0.798077	0.817308	0.817308	0.788462	0.798077	0.826923

   - For n = 4

```
Selected Feature Names:
('magnitude', 'sig', 'gap', 'depth')
DecisionTreeClassifier(max_features='sqrt', random_state=0)
Selected Feature Names:
('magnitude', 'sig', 'gap', 'depth')
```

	Logistic	SVML	KNN	NaiveBayes	DecisionTree	RandomForest
RandomForest	0.807692	0.817308	0.788462	0.759615	0.798077	0.807692
DecisionTree	0.807692	0.817308	0.788462	0.759615	0.798077	0.807692

   - For n = 5

```
Selected Feature Names:
('magnitude', 'sig', 'dmin', 'gap', 'depth')
DecisionTreeClassifier(max_features='sqrt', random_state=0)
Selected Feature Names:
('magnitude', 'sig', 'dmin', 'gap', 'depth')
```

	Logistic	SVML	KNN	NaiveBayes	DecisionTree	RandomForest
RandomForest	0.807692	0.807692	0.778846	0.759615	0.769231	0.788462
DecisionTree	0.807692	0.807692	0.778846	0.759615	0.769231	0.788462

Out of the four feature selection method, to select 3 features, forward selection and backward elimination provides best results and the selected features are: ‘magnitude’, ‘sig’, and ‘depth’.

IDENTIFYING BEST MODEL:

To find the best fitting parameters of each algorithms, GridSearchCV from sklearn is used. The results for parameters – ‘magnitude’, ‘sig’, ‘dmin’, ‘gap’, ‘depth’ are as follows:

Github: https://github.com/AuxiliaV/Week-9.1_MachineLearningAndDataScienceCapstoneProject/blob/main/EarthquakeAlertPrediction_ModelIdentification.ipynb

1. **Logistic Regression:**
   The Best Parameters are: {'C': 0.1, 'penalty': 'l2', 'random_state': 0, 'solver': 'lbfgs'} 0.7693735676088617

```
cm
```

```
array([[79,  1,  0,  1],
       [12,  1,  0,  1],
       [ 3,  0,  1,  0],
       [ 0,  1,  0,  4]], dtype=int64)
```

```
print(report)
```

```
              precision    recall  f1-score   support

           0       0.84      0.98      0.90        81
           1       0.33      0.07      0.12        14
           2       1.00      0.25      0.40         4
           3       0.67      0.80      0.73         5

    accuracy                           0.82       104
   macro avg       0.71      0.52      0.54       104
weighted avg       0.77      0.82      0.77       104
```

```
F1
```

```
0.7693735676088617
```

2. **Support Vector Machine:**
   The Best Parameters are: {'C': 10.0, 'kernel': 'linear'} 0.7905079307751678

```
cm
```

```
array([[77,  2,  2,  0],
       [11,  3,  0,  0],
       [ 0,  3,  1,  0],
       [ 0,  1,  1,  3]], dtype=int64)
```

```
print(report)
```

```
              precision    recall  f1-score   support

           0       0.88      0.95      0.91        81
           1       0.33      0.21      0.26        14
           2       0.25      0.25      0.25         4
           3       1.00      0.60      0.75         5

    accuracy                           0.81       104
   macro avg       0.61      0.50      0.54       104
weighted avg       0.78      0.81      0.79       104
```

```
F1
```

```
0.7905079307751678
```

3. **Decision Tree Classifier:**
   The Best Parameters are: {'criterion': 'gini', 'max_features': 'sqrt', 'random_state': 0, ' splitter': 'random'} 0.7895426412667792

```
cm
```

```
array([[76,  5,  0,  0],
       [ 6,  5,  3,  0],
       [ 1,  2,  1,  0],
       [ 1,  3,  0,  1]], dtype=int64)
```

```
print(report)
```

```
              precision    recall  f1-score   support

           0       0.90      0.94      0.92        81
           1       0.33      0.36      0.34        14
           2       0.25      0.25      0.25         4
           3       1.00      0.20      0.33         5

    accuracy                           0.80       104
   macro avg       0.62      0.44      0.46       104
weighted avg       0.81      0.80      0.79       104
```

```
F1
```

```
0.7895426412667792
```

4. **Random Forest Classifier:**
   The Best Parameters are: {'criterion': 'entropy', 'max_features': 'sqrt', 'n_estimators': 100, 'random_state': 45} 0.8082750582750583

```
cm
```

```
array([[77,  2,  2,  0],
       [ 8,  4,  2,  0],
       [ 2,  1,  1,  0],
       [ 0,  0,  2,  3]], dtype=int64)
```

```
print(report)
```

```
              precision    recall  f1-score   support

           0       0.89      0.95      0.92        81
           1       0.57      0.29      0.38        14
           2       0.14      0.25      0.18         4
           3       1.00      0.60      0.75         5

    accuracy                           0.82       104
   macro avg       0.65      0.52      0.56       104
weighted avg       0.82      0.82      0.81       104
```

```
F1
```

```
0.8082750582750583
```

5. **K Neighbors Classifier:**
   The Best Parameters are: {'algorithm': 'auto', 'n_neighbors': 6, 'weights': 'uniform'} 0 .7884780881768834

```
cm
```

```
array([[77,  3,  1,  0],
       [ 8,  4,  2,  0],
       [ 0,  4,  0,  0],
       [ 0,  3,  0,  2]], dtype=int64)
```

```
print(report)
```

```
              precision    recall  f1-score   support

           0       0.91      0.95      0.93        81
           1       0.29      0.29      0.29        14
           2       0.00      0.00      0.00         4
           3       1.00      0.40      0.57         5

    accuracy                           0.80       104
   macro avg       0.55      0.41      0.45       104
weighted avg       0.79      0.80      0.79       104
```

```
F1
```

```
0.7884780881768834
```

6. **Gaussian NB:**
   The Best Parameters are: {'var_smoothing': 1e-11} 0.7678571428571429

```
cm
```

```
array([[71,  5,  5,  0],
       [ 8,  5,  1,  0],
       [ 2,  1,  1,  0],
       [ 0,  1,  2,  2]], dtype=int64)
```

```
print(report)
```

```
              precision    recall  f1-score   support

           0       0.88      0.88      0.88        81
           1       0.42      0.36      0.38        14
           2       0.11      0.25      0.15         4
           3       1.00      0.40      0.57         5

    accuracy                           0.76       104
   macro avg       0.60      0.47      0.50       104
weighted avg       0.79      0.76      0.77       104
```

```
F1
```

```
0.7678571428571429
```

FOR THE SELECTED FEATURES: “magnitude”, “sig”, “depth”:

1. **Logistic Regression:**
   The Best Parameters are: {'C': 10.0, 'penalty': 'l2', 'random_state': 0, 'solver': 'lbfgs'}
   0.79551563202879

```
cm
```

```
array([[78,  1,  2,  0],
       [11,  3,  0,  0],
       [ 1,  2,  1,  0],
       [ 0,  0,  2,  3]], dtype=int64)
```

```
print(report)
```

```
              precision    recall  f1-score   support

           0       0.87      0.96      0.91        81
           1       0.50      0.21      0.30        14
           2       0.20      0.25      0.22         4
           3       1.00      0.60      0.75         5

    accuracy                           0.82       104
   macro avg       0.64      0.51      0.55       104
weighted avg       0.80      0.82      0.80       104
```

```
F1
```

```
0.79551563202879
```

2. **Support Vector Machine:**
   The Best Parameters are: {'C': 10.0, 'kernel': 'linear'} 0.7879293500617031

```
cm
```

```
array([[77,  2,  2,  0],
       [11,  3,  0,  0],
       [ 1,  2,  1,  0],
       [ 0,  1,  1,  3]], dtype=int64)
```

```
print(report)
```

```
              precision    recall  f1-score   support

           0       0.87      0.95      0.91        81
           1       0.38      0.21      0.27        14
           2       0.25      0.25      0.25         4
           3       1.00      0.60      0.75         5

    accuracy                           0.81       104
   macro avg       0.62      0.50      0.54       104
weighted avg       0.78      0.81      0.79       104
```

```
F1
```

```
0.7879293500617031
```

3. **Decision Tree Classifier:**
   The Best Parameters are: {'criterion': 'gini', 'max_features': 'sqrt', 'random_state': 0, ' splitter': 'best'} 0.8348381801125703

```
cm
```

```
array([[76,  3,  2,  0],
       [ 7,  6,  1,  0],
       [ 0,  2,  2,  0],
       [ 0,  2,  0,  3]], dtype=int64)
```

```
print(report)
```

```
              precision    recall  f1-score   support

           0       0.92      0.94      0.93        81
           1       0.46      0.43      0.44        14
           2       0.40      0.50      0.44         4
           3       1.00      0.60      0.75         5

    accuracy                           0.84       104
   macro avg       0.69      0.62      0.64       104
weighted avg       0.84      0.84      0.83       104
```

```
F1
```

```
0.8348381801125703
```

4. **Random Forest Classifier:**
   The Best Parameters are: {'criterion': 'gini', 'max_features': 'sqrt', 'n_estimators': 50, ' random_state': 45} 0.8158705265322912

```
cm
```

```
array([[79,  2,  0,  0],
       [ 9,  3,  2,  0],
       [ 1,  1,  2,  0],
       [ 0,  2,  0,  3]], dtype=int64)
```

```
print(report)
```

```
              precision    recall  f1-score   support

           0       0.89      0.98      0.93        81
           1       0.38      0.21      0.27        14
           2       0.50      0.50      0.50         4
           3       1.00      0.60      0.75         5

    accuracy                           0.84       104
   macro avg       0.69      0.57      0.61       104
weighted avg       0.81      0.84      0.82       104
```

```
F1
```

```
0.8158705265322912
```

5. **K Neighbors Classifier:**
   The Best Parameters are: {'algorithm': 'auto', 'n_neighbors': 6, 'weights': 'uniform'} 0.7963635914983219

```
cm
```

```
array([[78,  1,  2,  0],
       [ 8,  3,  3,  0],
       [ 0,  3,  1,  0],
       [ 0,  3,  0,  2]], dtype=int64)
```

```
print(report)
```

```
              precision    recall  f1-score   support

           0       0.91      0.96      0.93        81
           1       0.30      0.21      0.25        14
           2       0.17      0.25      0.20         4
           3       1.00      0.40      0.57         5

    accuracy                           0.81       104
   macro avg       0.59      0.46      0.49       104
weighted avg       0.80      0.81      0.80       104
```

```
F1
```

```
0.7963635914983219
```

6. **Gaussian NB:**
   The Best Parameters are: {'var_smoothing': 1e-07} 0.7844599844599844

```
cm
```

```
array([[73,  5,  3,  0],
       [ 9,  4,  1,  0],
       [ 2,  1,  1,  0],
       [ 0,  0,  1,  4]], dtype=int64)
```

```
print(report)
```

```
              precision    recall  f1-score   support

           0       0.87      0.90      0.88        81
           1       0.40      0.29      0.33        14
           2       0.17      0.25      0.20         4
           3       1.00      0.80      0.89         5

    accuracy                           0.79       104
   macro avg       0.61      0.56      0.58       104
weighted avg       0.79      0.79      0.78       104
```

```
F1
```

```
0.7844599844599844
```

Based on the above reports, the better performing algorithm with best parameters and better scores in comparative to the other algorithms, and features:

- Algorithm: Decision Tree Classifier.
- Features: ‘magnitude’, ‘sig’, and ‘depth’.
- Best parameters: ('criterion': 'gini'), ('max_features': 'sqrt'), ('random_state': 0), ('splitter': 'best')
- Accuracy: 84%

CONCLUSION:

The final model is created using Decision Tree Classifier with hyper-tuning paramet ers set to its best. The final model is dumped in “.sav” format, and in deployment it is loaded using “pickle” library. Three inputs: magnitude, significance and depth are received from the user and its corresponding “ALERT” result is displayed.

Github Model Creation: https://github.com/AuxiliaV/Week-9.1_MachineLearningAndDataScienceCapstoneProject/blob/main/EarthQuakeAlertPrediction_ModelCreation.ipynb

Github Deployment: https://github.com/AuxiliaV/Week-9.1_MachineLearningAndDataScienceCapstoneProject/blob/main/Deployment_EarthquakeAlertPrediction.ipynb